

Session 2: Persistent Data Structures

Block 1 — Persistent data structures

Léon Gondelman

September 9, 2026

SW7 Programming Paradigms Course

Aalborg University @ Copenhagen

Roadmap for today

Lecture.

- *Persistent data structures*: ephemeral versus persistent structures, the checkpoint trick and the case in which it fails, immutability $\Rightarrow$ safe sharing $\Rightarrow$ persistence, in Java and in Python.
- *A first look at OCaml*: a language in which persistence is the default, and in which the distinction is visible *in the types*.

Lab. We build a *persistent queue* from two lists and use it to explore a maze. It holds the *frontier* — the cells found but not yet visited — and does two jobs at once.

- **It chooses the search.** With the queue the maze is explored **breadth-first**; swap in today's persistent *stack* and the same code explores **depth-first**, unchanged.
- **It keeps the history.** No version is ever destroyed, so every earlier frontier still exists. We can step back through the search and ask what it held at earlier steps, having recorded nothing.

Block 1: Persistent Data Structures

Data structure = data + structural relations.

Computer programs usually operate on tables of information. In most cases these tables are not simply amorphous masses of numerical values; they involve important structural relationships between the data elements. In order to use a computer properly, we need to understand the structural relationships present within data, as well as the basic techniques for representing and manipulating such structure within a computer.

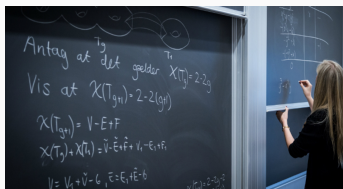
D. E. Knuth, The Art of Computer Programming, vol 1, chap 2,
“Information structures”, 1968.

An algorithm is like a cooking recipe!



- Input data and intermediate results are not preserved during computation.
- Fundamental formalism: the Turing machine.

An algorithm is a mathematical definition



- Computation seen as a sequence of steps constructing the final result, without ever erasing inputs or intermediate results.
- Fundamental formalisms: general recursive functions, λ -calculus, rewriting systems, formal semantics.
- “Declarative” programming (as opposed to “imperative”): functional, logic, constraint-based, etc.

What algorithms for declarative programming?

- Declarative programming is reputed inefficient because it cannot use *ephemeral* (in-place modified) data structures (e.g. arrays).
- **Persistent data structures** address this criticism:
 - Their interface only exposes operations that do not modify the structure in place, but instead return **new** modified structures.
 - Their implementation achieves, or comes close to, the complexity of the best known ephemeral structures.

The relevance of persistence for imperative programming

Even in imperative programming and classical algorithms, persistent data structures can be relevant:

- **Backtracking** (paths exploration, checkpoints) are trivial.
- One can keep **the complete history of updates** of the structure.
- The absence of in-place modification allows **memory sharing** between successive versions, enabling a compact representation of history.
- **concurrency** (e.g. immutable versions can be shared across threads without locks)

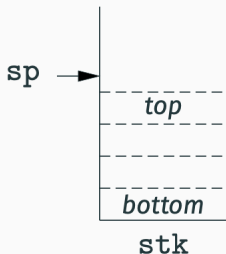
Example: A stack



Operations:

- `init` empties the stack
- `push(v)` pushes `v` on the top
- `top` returns the value at the top
- `pop` removes the value at the top

An ephemeral stack implemented with an array



```
int stk[SIZE];
int sp;

void init(void) { sp = 0; }

void push(int v) {
    assert (sp < SIZE);
    stk[sp] = v; sp = sp + 1;
}

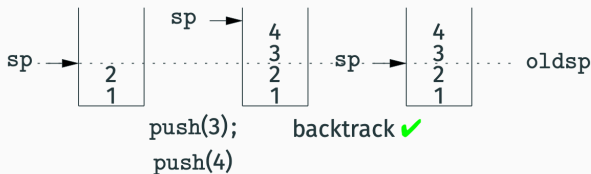
int top(void) {
    assert (sp > 0); return stk[sp - 1];
}

void pop(void) {
    assert (sp > 0); sp = sp - 1;
}
```

Backtracking (checkpointing & backtracking)

Efficient but incomplete approach:

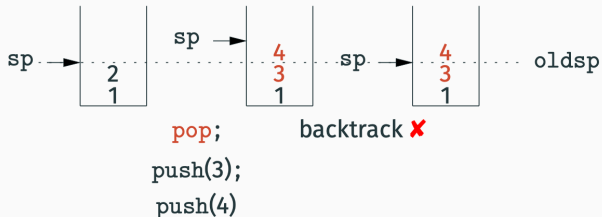
- Checkpoint: `oldsp = sp` (save the stack pointer)
- Backtrack: `sp = oldsp` (restore the stack pointer)



Backtracking (checkpointing & backtracking)

Efficient but incomplete approach:

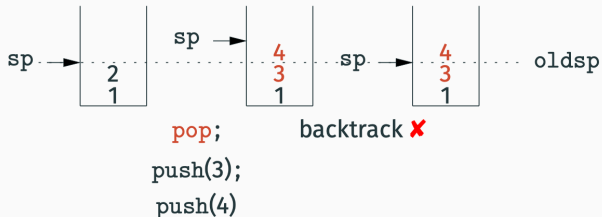
- Checkpoint: `oldsp = sp` (save the stack pointer)
- Backtrack: `sp = oldsp` (restore the stack pointer)



Backtracking (checkpointing & backtracking)

Efficient but incomplete approach:

- Checkpoint: `oldsp = sp` (save the stack pointer)
- Backtrack: `sp = oldsp` (restore the stack pointer)



Always correct but inefficient alternative:
copy `stk[0..sp]` to another array.

A persistent stack implemented as a linked list



```
final class Stack {  
    private final int hd;  
    private final Stack tl;  
    private Stack(int v, Stack s) { hd = v; tl = s; }  
  
    static final Stack empty = null;  
    static Stack push(int v, Stack s) { return new Stack(v, s); }  
    static int top (Stack s) { return s.hd; }  
    static Stack pop (Stack s) { return s.tl; }  
}
```

The interface of the data structure changes: push and pop *return* the new stack value (no in-place mutation).

The implementation relies on the language memory management (GC) to reclaim list cells after pop.

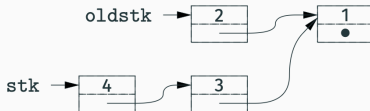
Note every `private` and every `final`. We come back to why.

Persistent stacks and backtracking



```
stk = Stack.push(2, Stack.push(1, Stack.empty));
```

Persistent stacks and backtracking



```
stk = Stack.push(2, Stack.push(1, Stack.empty));
```

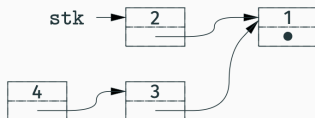
```
// Checkpoint
```

```
oldstk = stk;
```

```
// Work
```

```
stk = Stack.push(4, Stack.push(3, Stack.pop(stk)));
```

Persistent stacks and backtracking



```
stk = Stack.push(2, Stack.push(1, Stack.empty));
```

```
// Checkpoint
```

```
oldstk = stk;
```

```
// Work
```

```
stk = Stack.push(4, Stack.push(3, Stack.pop(stk)));
```

```
// Backtrack
```

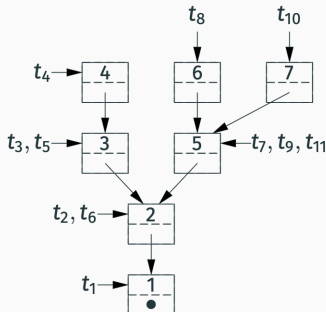
```
stk = oldstk;
```

Persistent stacks and memory sharing

Key observation: a stack produced by push or pop **shares** all but one node with the previous stack.

This enables keeping *all* successive states economically, so that total memory for a sequence of N pushes and M pops is about N nodes.

```
t1 = push(1, empty)
t2 = push(2, t1)
t3 = push(3, t2)
t4 = push(4, t3)
t5 = pop(t4)
t6 = pop(t5)
t7 = push(5, t6)
t8 = push(6, t7)
t9 = pop(t8)
t10 = push(7, t9)
t11 = pop(t10)
```



Why private? Why final?

Two keywords we put on the Java linked-list stack, and never justified.

```
Stack v1 = push(3, push(2, push(1, empty))); // [3, 2, 1]
Stack v2 = pop(v1);                          // [2, 1]
        // v2 IS the second node of v1 -- the two versions SHARE it

v2.tl = null; // if tl were public and non-final
```

What is v1 now?

Immutability, Sharing, Persistence

- **Immutability:** operations never overwrite state; they *return a new value*.
- **Structural sharing:** a new version *reuses the unchanged parts* of the old one — a new node whose tail *is* the old stack.
- **Safe sharing:** no version can be changed *through another version's pointer*. Sharing on its own is merely aliasing — the previous slide. *Immutability buys this outright*, by making the change impossible.
- **Persistence:** every old version stays valid and reachable.

immutability $\Rightarrow$ safe sharing $\Rightarrow$ persistence

An implication, not an equivalence — next slide.

Immutability is one route, not the only one

Safe sharing asks for one thing: *no version may be changed through another version's pointer*. There is more than one way to arrange that.

Route	Safe because	Checked by
Immutability	nothing ever changes	compiler: <code>final</code> , <code>frozen</code>
Encapsulation	only the owner may change it	compiler, for clients
Ownership	aliasing and mutation never meet	compiler (Rust)
Benign effects	it changes, nobody can tell	us, by argument
Copy-on-write	a writer gets its own copy	the runtime

Today we take immutability: the compiler checks it, and it preserves *structural* sharing. Copy-on-write is just as safe, but duplicates the whole structure at the first write — the very thing we are avoiding.

Later we will need benign effects. Session 5 repairs today's queue with a thunk that overwrites itself once forced; session 7 compresses paths in a persistent Union-Find. Both mutate, and no client can tell — which is why that row is checked by argument rather than by a compiler.

If immutability or safe sharing is missing

Sharing without immutability $\Rightarrow$ *corruption*.

```
v2.t1 = null;    // v1 was [3,2,1] -- now [3,2]
```

An old version changed although nobody touched it. Restore that checkpoint while backtracking, and we resume from a state that never existed.

Immutability without sharing $\Rightarrow$ *copying*. To keep a checkpoint we must copy the whole structure: a stack of 10^6 nodes costs 10^6 nodes per checkpoint instead of one pointer. Backtracking degrades from $O(1)$ to $O(n)$ per step.

We need both: drop either one and persistence becomes either *wrong* or *unaffordable*.

Java: why `private` and `final`?

Back to the open question: `v2.tl = null` does not compile. That is the whole point.

- `private` fields: encapsulates representation — clients cannot mutate internals.
- `final` **fields**: enforce immutability after construction $\Rightarrow$ not even Stack itself can rewrite a node once it exists, and the compiler checks it.
- `final` **class** (optional but recommended): prevents subclassing that could reintroduce mutability or break invariants.

Bonus, and a session 7 teaser: `final` also makes these fields safely visible to other threads, with no locking.

Python: tuples-as-cons — immutability & sharing

In Python once tuples are created, their items can't be updated.
Stack as nested tuples: (hd, tl), where tl is another pair or None.

```
EMPTY = None
```

```
def push(v, s):      # O(1)
    return (v, s)    # NEW tuple; 's' is SHARED (structural sharing)
```

```
def top(s):
    if s is None: raise IndexError("empty")
    return s[0]
```

```
def pop(s):          # O(1)
    if s is None: raise IndexError("empty")
    return s[1]
```

```
# Example: versions share structure
```

```
s1 = push(1, EMPTY)      # [1]
s2 = push(2, s1)         # [2,1] (shares node '1')
s3 = push(3, s2)         # [3,2,1] (shares tail [2,1])
```

```
# s2[1] = None          # -> TypeError (tuples are immutable)
```

Sharing: push creates 1 new pair & reuses the whole tail $\Rightarrow O(1)$ upd.

Python: immutable cons-list (structural sharing)

A persistent stack needs nodes that can never change once built. Python says so in one word: frozen.

```
from dataclasses import dataclass
from typing import Optional

@dataclass(frozen=True)
class Stack:
    hd: int
    tl: "Optional[Stack]" = None

def push(v, s): return Stack(v, s)    # one new node; tail s is SHARED
def pop(s):
    if s is None: raise IndexError("empty")
    return s.tl

s1 = push(1, None); s2 = push(2, s1); s3 = push(3, s2)
s2.tl = None           # FrozenInstanceError -- s3 is protected
```

Same story as in Java, one line shorter: `frozen=True` generates the `__setattr__` that refuses the write. Without it, `s2.tl = None` would retroactively change `s3`, which shares `s2` as its tail.

Some languages make persistence the default

Everything so far was persistence obtained *against the language's defaults*: Java needed `private` and `final`, Python needed `frozen`. Some languages default the other way.

Lisp / Scheme (1958 onwards) — the cons cell is immutable by convention, and the whole language is built on it:

```
(define s1 (cons 1 '()))      ; [1]
(define s2 (cons 2 s1))      ; [2,1] -- s1 is SHARED, not copied
```

Haskell (1990) — a *pure* language: ordinary code has no assignment at all, so every structure is persistent whether we intended it or not. Mutation exists, but only where the type admits it:

```
s1 = [1]
s2 = 2 : s1    -- the same cons, spelled with a colon
```

The same data structure, and no keyword required: the default has done the work for us.

OCaml: the stack, one more time

Our language for the remainder of the course. Immutable by default, yet mutation costs nothing to reach for — where Haskell makes us say so in the type. Both languages let us *choose*; they differ in what the choice costs, and that is one of the subjects of this course.

The persistent stack, in five lines:

```
type 'a t = 'a list

let empty = []
let push v s = v :: s
let pop s = match s with [] -> None | v :: rest -> Some (v, rest)
let peek s = match s with [] -> None | v :: _ -> Some v
```

Reading it: `let` binds a name; `'a` is Java's `<T>`; `[]` is the empty list and `v :: s` puts `v` in front of `s`; `match` performs case analysis on the shape of a value — the switch statement we always wanted.

`v :: s` *is* the sharing diagram we drew earlier — one new cell, and the old stack reused entire.

The persistence material of this lecture follows
Xavier Leroy's course at the Collège de France.

The amortization and queue material follows
Chris Okasaki, *Purely Functional Data Structures*,
Cambridge University Press, 1998.